

FINAL PROJECT REPORT

AI HOUSE FINDER

A Personalized Rental Recommendation Agent Using LangGraph, LLM-Driven Preference Learning, and Iterative Feedback Loops

Shreyas Pantangi · Kanishk Deshwal · Rakshit Shetty

Rutgers University · Artificial Intelligence & Large Language Models · Spring 2026

GitHub: github.com/deshwal88/AI-House-Hunting-Agent

ABSTRACT

Searching for a rental home today means wading through hundreds of listings on platforms like Zillow or Apartments.com, applying blunt keyword filters, and manually evaluating each result — a process that captures what the user typed, not what they actually value. This project builds an AI House Finder: a stateful, LLM-powered agent that replaces keyword filtering with genuine preference learning. The system uses LangGraph to model an eight-node directed pipeline with a feedback loop. Google Gemini interprets a user's free-text comments to extract latent preferences, scores all candidate properties against a dynamically maintained preference profile, and rewrites its own scoring weights after each round of user feedback — without ever re-fetching properties from the network. Properties are sourced from the RentCast API and enriched with real-time amenity proximity data from the ArcGIS Places API. Scoring is divided into a fixed-weight hard scorer (price, bedrooms, bathrooms, area efficiency) and a learnable soft scorer (grocery distance, school proximity, gym access, transit, pet-friendliness, area, price value). The final score starts at a 60/40 hard-soft blend and shifts to 30/70 once feedback is received, letting learned preferences dominate after the first interaction. The feedback loop converges under one of three conditions: the user's drag-and-drop ordering matches the system ranking, the cosine similarity between consecutive soft-weight vectors exceeds 0.97, or a maximum of three refinement rounds is reached. The result is a system that learns the user's taste interactively, rather than requiring them to express it perfectly upfront.

1. HOW THIS DIFFERS FROM TRADITIONAL PLATFORMS LIKE ZILLOW

The motivation for this project begins with a fundamental limitation of every major rental search platform in use today. Zillow, Apartments.com, Trulia, and their peers are, at their core, *database query engines* with a well-designed UI layered on top. A user specifies a city, a price range, a bedroom count, and perhaps one or two

amenity checkboxes; the platform returns everything in its database that satisfies those predicates. The user is then left to manually evaluate dozens or hundreds of results — reading descriptions, eyeballing photos, and trying to remember which one had the closer grocery store.

This model has three structural weaknesses that no amount of UI polish can fully overcome:

Dimension	Traditional Platforms (Zillow et al.)	AI House Finder
Input model	Explicit keyword filters and range sliders. The user must know exactly what they want and express it in categorical terms.	Free-text comments plus structured form. The LLM infers latent preferences the user never stated explicitly (e.g. "I have a dog" → pet-friendly + possibly ground-floor preference).
Ranking	Listings are sorted by recency, price, or a black-box "Featured" score that typically reflects advertiser spend — not the user's preferences.	Properties are ranked by a computed score that blends objective criteria (price match, bedroom fit, area efficiency) with learnable preference features (amenity proximity, pet-friendliness, space).
Personalization	Static. The search resets after each session. Repeated searching is required with manually adjusted filters. There is no memory of what the user evaluated or rejected.	Dynamic. After the user drags and drops results into their preferred order, the system infers which features they care most about and adjusts the scoring weights before re-ranking — within the same session.
Preference representation	None. The platform has no model of the user's taste; it only knows the filters they set.	The system maintains a natural-language preference profile, updated by the LLM after each feedback round, describing the user's priorities in prose (e.g. "prioritises walkable grocery access and a quiet street over gym proximity").
Amenity data	Coarse proximity badges ("Near parks", "Walkable") based on zip-code level data, if present at all.	Property-level distances in miles to the nearest grocery store, school, gym, and bus stop, computed in real time from GPS coordinates via the ArcGIS Places API.
Explanation	None. The user does not know why one listing appears above another.	Every property card shows a one-to-two sentence LLM-generated rationale explaining why it fits the user's stated preferences.
Feedback mechanism	Implicit (save, contact, view time). No explicit preference signal is captured or acted upon.	Explicit drag-and-drop reordering. The agent interprets the gap between system ranking and user ranking to update soft-score weights for the next iteration.
Architecture	Request-response web app. Each search is independent; no stateful processing.	Stateful directed graph (LangGraph) with a feedback loop. State persists across iterations within a session; the graph loops back to re-score without re-fetching properties from the API.

Table 1. Architectural and functional comparison between traditional rental search platforms and the AI House Finder.

Key distinction: Zillow answers the question "Which listings match my filters?" The AI House Finder answers a fundamentally different question: "Which listings will I actually prefer — even if I cannot fully articulate why?"

This shift from filter-matching to preference-learning is what motivates every architectural decision in this project: the choice of LangGraph over a linear pipeline, the dual hard/soft scoring model, the LLM-driven weight update mechanism, and the convergence framework. Each of these is described in the sections that follow.

2. INTRODUCTION

Rental search is, for most people, an episodic process of manual labor. A prospective tenant opens Zillow, sets a rent range, picks a city, and scrolls through dozens of results with no guidance on which matter most for their specific situation. The system does not know that they work from home and need quiet, that they have a dog, that they walk to the grocery store every day, or that transit proximity matters less to them than school district quality. It cannot know, because it never asks — and even if it did, it would have no mechanism to act on the answers in a sophisticated way.

Large Language Models change this calculus. An LLM can read a user's free-text description of their lifestyle, infer unstated preferences, generate natural-language rationales for each candidate property, and most importantly, interpret the *signal* embedded in a user's preference ordering without requiring them to fill out a detailed preference questionnaire. The challenge is embedding the LLM inside a production-grade pipeline that fetches real listing data, enriches it with geocoded amenity information, and loops through user feedback without wasting API calls on re-fetching data that has not changed.

This report documents the design and implementation of such a system. The AI House Finder is built around three core contributions. First, a dual-scoring architecture separates objective property criteria (handled with fixed rule-based weights) from preference-based criteria (handled with LLM-learnable weights). Second, a LangGraph stateful graph models the feedback loop as a conditional directed graph, enabling re-scoring without re-fetching. Third, a FastAPI asynchronous bridge decouples the LLM agent from the Streamlit UI, allowing the graph to pause and await human feedback via an `asyncio.Event` mechanism while the UI remains responsive.

3. SYSTEM ARCHITECTURE

3.1 High-Level Design

The system consists of three loosely coupled layers: a Streamlit frontend that collects user requirements and displays results, a FastAPI backend that bridges the frontend to the agent, and a LangGraph agent that executes the

eight-node processing pipeline. The layers communicate exclusively via HTTP, keeping them independently testable and deployable.

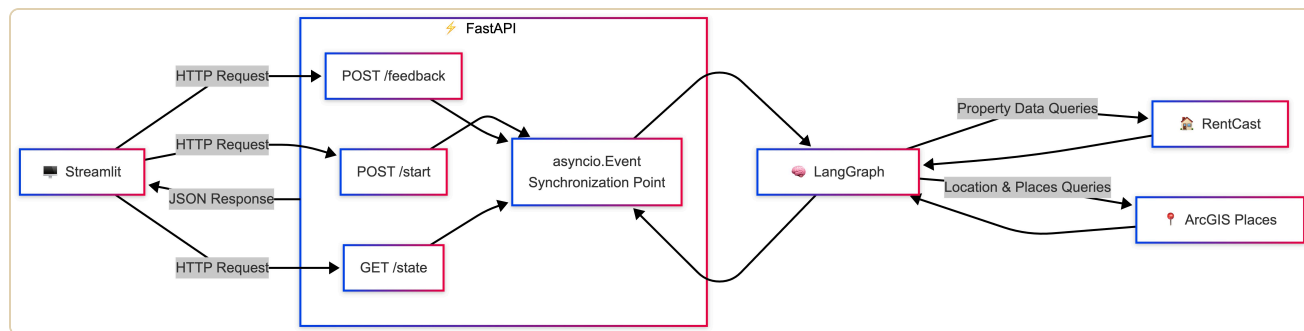


Figure 1. Three-layer system architecture. The Streamlit frontend communicates with the LangGraph agent exclusively through FastAPI HTTP endpoints.

3.2 Framework Choice: LangGraph

A natural question is why LangGraph was chosen over a simpler linear LangChain pipeline or plain Python. The answer is the feedback loop. A linear chain executes once and terminates. This system must loop: display results, receive user feedback, update scoring weights, re-rank the *same* properties, and display results again — repeating until convergence. Managing this loop manually in Python requires explicit state tracking, loop control logic, and conditional branching that must be wired together by hand. LangGraph models this directly as a stateful directed graph with typed edges and conditional routing.

Concretely, the feedback loop in this system re-enters the graph at Node 5 (soft scoring) — not at Node 2 (property fetching). Properties are fetched only once per session; only their scores and ordering change across iterations. LangGraph’s conditional edges make this re-entry point explicit and inspectable, whereas a plain-Python loop would bury this decision inside control flow logic.

3.3 LLM Choice: Google Gemini

The system uses two Gemini models, selected for their respective roles. Nodes that require richer reasoning — the requirement ingestion and the batch rationale generation — use `gemini-3-flash-preview`. The weight update node, which involves structured JSON output and must run after every feedback round, uses `gemini-3.1-flash-lite-preview` for lower latency. Both models are accessed through LangChain’s `ChatGoogleGenerativeAI` interface, which means the model string is the only line that would need to change to switch to GPT-4o or Claude.

The LLM is a *core architectural component* here, not a helper utility. It participates in three of the eight pipeline nodes, each with a different role: natural-language understanding (Node 1), preference-aligned scoring rationale generation (Node 5), and structured preference weight inference from implicit feedback signals (Node 8).

4. THE AGENT STATE

All information produced or consumed by any node in the pipeline flows through a single typed object: `AgentState`. This `TypedDict` enforces a contract between nodes — each node reads only from well-defined fields and writes only to fields it owns. The state is initialized at the start of each session and persists in memory across all feedback iterations.

```
class AgentState(TypedDict):
    # — Input —————
    requirements: dict          # structured dict from Streamlit wizard
    session_id: str

    # — After Node 2: fetch_properties —————
    properties: List[dict]      # normalized property dicts from RentCast

    # — After Node 3: enrich_properties —————
    enriched_properties: List[dict] # properties + amenity_distances

    # — After Node 4: hard_score —————
    hard_scores: Dict[str, float] # property_id -> 0.0-1.0

    # — After Node 5: soft_score (updated each loop) —————
    soft_scores: Dict[str, float] # property_id -> 0.0-1.0
    soft_rationales: Dict[str, str] # property_id -> LLM explanation

    # — After Node 6: rank_and_display —————
    ranked_list: List[dict]      # sorted properties with final_score

    # — After Node 7: collect_feedback —————
    user_feedback: List[str]     # ordered property_ids from drag-drop

    # — Learned state (evolves across feedback loops) —————
    soft_weights: Dict[str, float] # feature_name -> weight (sum to 1.0)
    preference_profile: str        # LLM natural-language summary

    # — Control flow —————
    iteration: int                # feedback loop counter, starts at 0
    converged: bool               # termination flag
    convergence_reason: str       # why it converged (shown in UI)
```

The state is stored in an in-memory dictionary keyed by session ID on the FastAPI server. Because each node function receives the full state dict and returns an updated copy, the graph execution is functionally transparent: at any point during processing, the Streamlit frontend can read the current state via the `GET /state/{session_id}` endpoint and display partial results.

5. THE EIGHT-NODE PIPELINE

The LangGraph agent is built around eight nodes wired into a directed graph. Nodes 1 and 2 run in parallel (they are independent: ingest reads requirements, fetch calls the RentCast API). All other nodes execute sequen-

tially. The feedback loop re-enters at Node 5, bypassing Nodes 1–4 entirely on subsequent iterations.

5.1 Node 1 — Requirement Ingestion (LLM)

The first node bridges the gap between what the user typed and what they actually mean. The Streamlit wizard produces a structured JSON object containing location, budget, bedroom count, bathrooms, property type, and boolean preferences. It also accepts an optional free-text comment field. The LLM (Gemini Flash) reads the entire requirements object and infers latent preferences the structured form cannot capture.

For example, a user who writes "I work from home and have a large dog" has not explicitly requested a quiet neighborhood or pet-friendly building — but both follow from the statement. The model is prompted to identify these implications and produce a concise natural-language preference profile (three to five sentences) that summarizes what matters most to the user. This profile is stored in `state["preference_profile"]` and used by every downstream LLM call as the user's "voice" in the system.

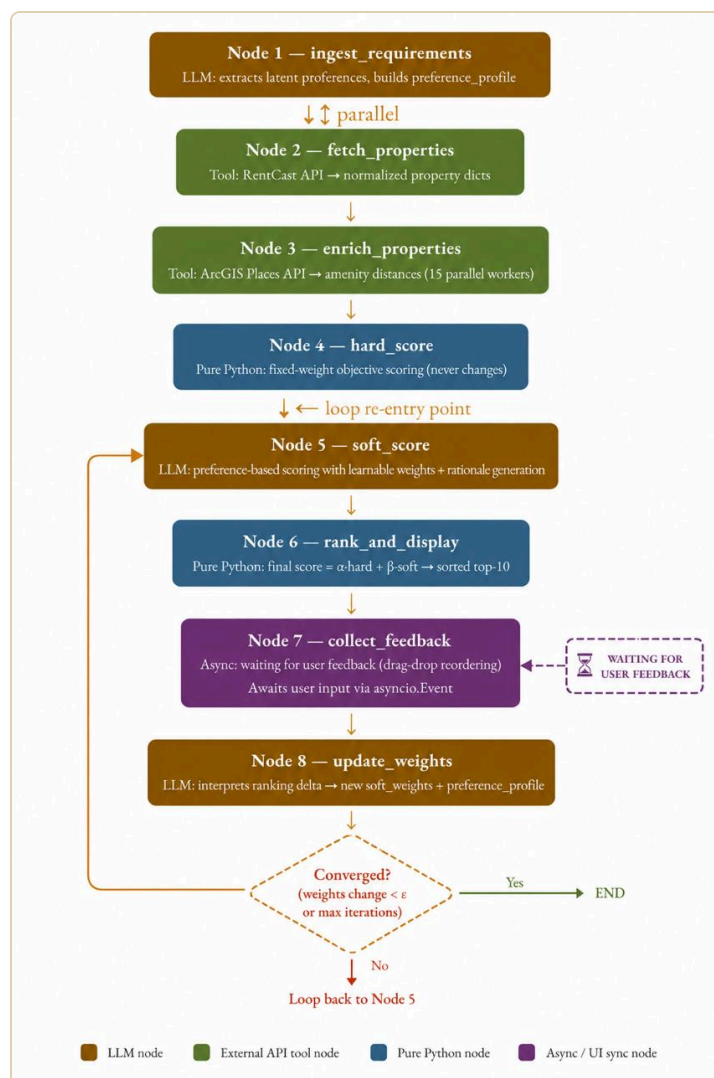


Figure 2. Eight-node LangGraph pipeline with feedback loop. Nodes 1 and 2 run in parallel. The loop re-enters at Node 5, not Node 2 — properties are fetched only once per session.

```

prompt = f"""You are a rental requirements analyst for a personalized home-finding AI.

The user completed a structured search form with these details:
{req}

{"They also added this free-text comment: " + repr(comments) if comments else "No additional comments."}

Your tasks:
1. Identify latent preferences implied by their inputs.
2. Write a concise preference profile (3–5 sentences) summarizing what matters most.

Return ONLY the preference profile. No preamble, no bullet points, no markdown."""

```

The node also initializes the soft weights to uniform (1/8 each across eight features), resets the iteration counter to zero, and clears all previously accumulated scores and feedback — ensuring a clean slate for each new search session.

5.2 Node 2 — *Property Fetch (RentCast API)*

Node 2 calls the RentCast `/v1/listings/rental/long-term` endpoint with the user's structured parameters: city, state, optional ZIP code, price range, bedroom count, bathroom count, and search radius. The fetch targets up to 20 candidates; if fewer than 20 are returned, the search radius is automatically expanded (2×, then 3× the user's specified radius) and retried. This radius-expansion strategy handles sparse markets where a narrow radius would otherwise return an insufficient pool of candidates.

Each returned listing is normalized into a standard dict containing: `property_id`, `address`, `city`, `state`, `zip_code`, `rent`, `bedrooms`, `bathrooms`, `sqft`, `property_type`, `latitude`, and `longitude`. Normalization ensures that all downstream nodes receive a consistent schema regardless of how RentCast structures its API response for any given market.

Nodes 1 and 2 are executed in parallel using `asyncio.gather`. Since both nodes only read from `state["requirements"]` and write to separate fields, they are data-independent and can safely run concurrently. This reduces Phase 1 wall-clock time by approximately the duration of the RentCast API call (typically 1–3 seconds).

5.3 Node 3 — *Enrichment (ArcGIS Places API)*

The enrichment node augments each property with real-time proximity data for four amenity categories: grocery stores, elementary schools, gyms, and bus stops. For each property, it sends four parallel HTTP requests to the ArcGIS Places `/places/near-point` endpoint — one per amenity type — using the property's latitude and longitude. Each request returns the nearest matching place within a 5,000-meter radius; the distance is converted from meters to miles.

```

AMENITY_SEARCHES = {
    "grocery": "grocery store",
    "school": "elementary school",
    "gym": "gym fitness",
    "transit": "bus stop",
}

# Four parallel searches per property
with ThreadPoolExecutor(max_workers=len(AMENITY_SEARCHES)) as pool:
    futures = {
        pool.submit(_nearest_distance_miles, lat, lng, search_text, token): amenity
        for amenity, search_text in AMENITY_SEARCHES.items()
    }

```

To keep enrichment time manageable, the node pre-filters the fetched properties down to 15 candidates before calling ArcGIS. The pre-filter ranks properties by proximity to the midpoint of the user's budget range (as a proxy for relevance), keeping only the 15 closest. All 15 enrichment calls are dispatched concurrently using a 15-worker `ThreadPoolExecutor`, so total enrichment wall-clock time is approximately equal to the time for a single ArcGIS call rather than the sum of all calls.

Properties for which ArcGIS returns no nearby result receive a default distance of 99.0 miles — a sentinel value that produces a soft score of zero for that amenity feature, ensuring the scoring logic handles missing data gracefully without special-casing.

5.4 Node 4 — Hard Scoring (Pure Python)

Hard scoring evaluates each property against objective, user-specified criteria using a fixed set of weights that never change across feedback iterations. The five features and their weights are:

Feature	Weight	Scoring Function
Price match	0.35	$\max(0, 1 - \text{rent} - \text{midpoint} / \text{midpoint})$, where $\text{midpoint} = (\text{price_min} + \text{price_max}) / 2$
Bedroom match	0.25	$\max(0, 1 - \text{actual} - \text{wanted} \times 0.5)$ — penalizes by 0.5 per bedroom difference
Bathroom match	0.15	Same structure as bedroom match
Sqft per dollar	0.15	$\text{sqft} / \text{rent}$, min-max normalized across all enriched candidates in the current batch
Within radius	0.10	Binary 1.0 (RentCast already filters by radius, so this is always 1.0 for returned listings)

Table 2. Hard scoring features, weights, and scoring functions. Weights sum to 1.0. These weights are fixed and never updated by user feedback.

The sqft-per-dollar normalization is computed across the current batch of enriched properties, not globally. This means the score reflects relative value within the candidate pool rather than against an absolute standard — a property at \$1,000/month for 800 sqft scores higher on this dimension than a \$3,000/month property at 1,500 sqft, even if the latter is larger in absolute terms.

5.5 Node 5 — Soft Scoring (LLM + Python)

Soft scoring is the personalization layer. It evaluates eight preference-based features with weights that start uniform (1/8 each) and are updated by the LLM after each feedback round. The eight features are:

Feature	Default Weight	Scoring Function
Grocery distance	1/8	<code>max(0, 1 - d / 2.0)</code> — decays linearly, zero at ≥ 2 miles
School distance	1/8	<code>max(0, 1 - d / 3.0)</code> — zero at ≥ 3 miles
Gym distance	1/8	<code>max(0, 1 - d / 2.0)</code> — zero at ≥ 2 miles
Transit distance	1/8	<code>max(0, 1 - d / 0.5)</code> — very tight decay; zero at ≥ 0.5 miles
Pet friendly	1/8	Binary 1.0 / 0.0
Quiet neighborhood	1/8	Proxy: <code>min(1, gym_distance / 2.0)</code> — farther from high-traffic areas implies quieter
Area (sqft)	1/8	Min-max normalized sqft across the current batch
Price value	1/8	<code>(price_max - rent) / (price_max - price_min)</code> — lower rent within budget scores higher

Table 3. Soft scoring features, default weights, and scoring functions. All weights must sum to 1.0. These weights are updated by the LLM after each feedback round.

The numeric soft scores are computed entirely in pure Python from the ArcGIS amenity distances and property attributes. The LLM is not called for scoring computation — it is called once per batch, in a single parallelized prompt, to generate natural-language rationales for all properties simultaneously:

```
prompt = f"""User preference profile: {preference_profile}

For each property below write at most 20 words explaining why it fits the user's preferences.
Be specific. No filler phrases like "This property offers".

Properties:
{chr(10).join(lines)}
```

```
Respond ONLY with a JSON object mapping each property_id to its rationale string.  
No markdown, no extra keys."""
```

Batching all rationale requests into a single LLM call — rather than one call per property — reduces API latency from $O(n)$ to $O(1)$ with respect to the number of properties. Rationales from previous iterations are cached in `state["soft_rationales"]`; only properties that appeared for the first time in the current iteration require new rationale generation.

5.6 Node 6 — Ranking

The rank node combines hard and soft scores into a final scalar, sorts all enriched properties in descending order, and retains the top 10 for display. The blend weights shift between iterations:

```
final_score = hard_weight × hard_score + soft_weight × soft_score  
  
# Before any feedback: hard_weight = 0.6, soft_weight = 0.4  
# After first feedback: hard_weight = 0.3, soft_weight = 0.7
```

The rationale for the 0.6/0.4 initial split is that on the first iteration the system has no learned signal about the user's taste, so objective criteria should anchor the ranking. After the user provides feedback and the system has inferred preference weights, soft scoring becomes the dominant signal — the 0.3/0.7 post-feedback split reflects this. This weighting shift is automatic and does not require explicit user configuration.

5.7 Node 7 — Feedback Collection (Async)

Node 7 is architecturally unusual: it is a no-op at the graph level. Its role is entirely synchronization. After Node 6 writes the ranked list to state, the FastAPI layer detects that results are ready and sets the `awaiting_feedback` flag in the state, which the Streamlit frontend polls every two seconds. The Streamlit UI then presents the drag-and-drop reordering interface.

The graph pauses here — not via a blocking sleep, but via an `asyncio.Event` stored in the `feedback_events` dict on the FastAPI server. When the user submits their reordering via the Streamlit "Submit My Ranking" button, Streamlit POSTs the ordered list of property IDs to the `/feedback` endpoint. The endpoint writes the list to `state["user_feedback"]` and calls `event.set()`, which unblocks the graph and allows it to proceed to Node 8.

```
@api.post("/feedback")  
async def receive_feedback(body: FeedbackInput):  
    agent_states[sid]["user_feedback"] = body.ordered_property_ids  
    if sid in feedback_events:  
        feedback_events[sid].set()          # unblocks the waiting graph  
        feedback_events[sid] = asyncio.Event() # reset for next round  
    return {"status": "received"}
```

5.8 Node 8 — Weight Update (LLM)

Node 8 is the preference learning engine. It receives the system's ranking (the order in which Node 6 sorted properties) and the user's ranking (the order the user dragged them into), constructs a side-by-side comparison, and asks the LLM to infer what the user values more and less than currently weighted.

```
prompt = f"""Current preference profile:
{state["preference_profile"]}

Current soft weights:
{json.dumps(state["soft_weights"], indent=2)}

System vs. user ranking comparison:
{chr(10).join(comparison_lines)}

Analyze the differences. Determine what amenities/features the user values MORE or LESS.
Update both the preference profile and the soft weights.

Respond in this exact JSON format:
{{
  "updated_preference_profile": "...",
  "updated_soft_weights": {{
    "grocery_distance": 0.0,    // must sum to 1.0
    "school_distance": 0.0,
    ...
  }},
  "reasoning": "one sentence explaining the key weight change"
}}
```

The LLM's output is parsed, the weights are renormalized to sum to 1.0 (guarding against floating-point drift), and both the preference profile and soft weights are written back to state. The preference profile thus evolves across iterations — each round of feedback refines the system's understanding of the user's taste in natural language.

6. CONVERGENCE MECHANISM

The feedback loop terminates under the first of three conditions that is satisfied:

Condition	Criterion	Interpretation
Order match	<code>system_ranking == user_ranking</code>	The user agrees with the system's ranking exactly — no more information is gained from further feedback.
Weight stability	<code>cosine_similarity(old_weights, new_weights) > 0.97</code>	The LLM's weight update produced weights so similar to the previous iteration that further refinement is unlikely to change the ranking meaningfully.

Condition	Criterion	Interpretation
Maximum iterations	<code>iteration ≥ 3</code>	Hard cap at three refinement rounds. Prevents infinite loops in production and ensures the session terminates predictably.

Table 4. Three convergence conditions, checked in order inside Node 8. The loop terminates when any condition is satisfied.

```
def weights_converged(old_weights, new_weights, threshold=0.97):
    old = np.array([old_weights.get(k, 0) for k in DEFAULT_SOFT_WEIGHTS])
    new = np.array([new_weights.get(k, 0) for k in DEFAULT_SOFT_WEIGHTS])
    norm_product = np.linalg.norm(old) * np.linalg.norm(new)
    if norm_product == 0:
        return False
    return float(np.dot(old, new) / norm_product) > threshold
```

Cosine similarity is chosen over Euclidean distance because it is invariant to the scale of the weight vectors. Since weights are renormalized to sum to 1.0 after each update, the vectors are already on the unit simplex; cosine similarity in this space measures the angle between weight allocations rather than their absolute difference. A similarity of 0.97 corresponds to a very small angular deviation, meaning the two weight vectors assign nearly identical relative importance to all eight features.

Upon convergence, the reason is written to `state["convergence_reason"]` and surfaced in the Streamlit UI as a success banner, closing the interaction loop with a human-readable explanation of why the agent stopped refining.

7. BACKEND: FASTAPI BRIDGE

Streamlit's execution model is synchronous and re-runs the entire script on every user interaction. LangGraph's agent is asynchronous and long-running. The two cannot be directly integrated; a bridge layer is required. FastAPI serves this role via three endpoints:

Endpoint	Called by	Purpose
<code>POST /start</code>	Streamlit (on form submit)	Initializes AgentState, creates an <code>asyncio.Task</code> to run the graph asynchronously, and returns immediately. The graph runs in the background.
<code>POST /feedback</code>	Streamlit (on drag-drop submit)	Writes the user's ordered property ID list to state and calls <code>event.set()</code> to unblock the graph at Node 7.
<code>GET /state/{session_id}</code>	Streamlit (polling every 2s)	Returns the current <code>ranked_list</code> , <code>preference_profile</code> , <code>iteration</code> , <code>converged</code> , <code>convergence_reason</code> , and <code>awaiting_feedback</code> flag.

Table 5. FastAPI endpoints bridging the Streamlit frontend and the LangGraph agent.

State is held in two in-memory Python dictionaries keyed by session ID: `agent_states` (the full AgentState for each session) and `feedback_events` (the `asyncio.Event` for each session). This design is suitable for single-worker development and small-scale deployment; a production system would replace the in-memory store with Redis to support multi-worker deployments and session persistence across server restarts.

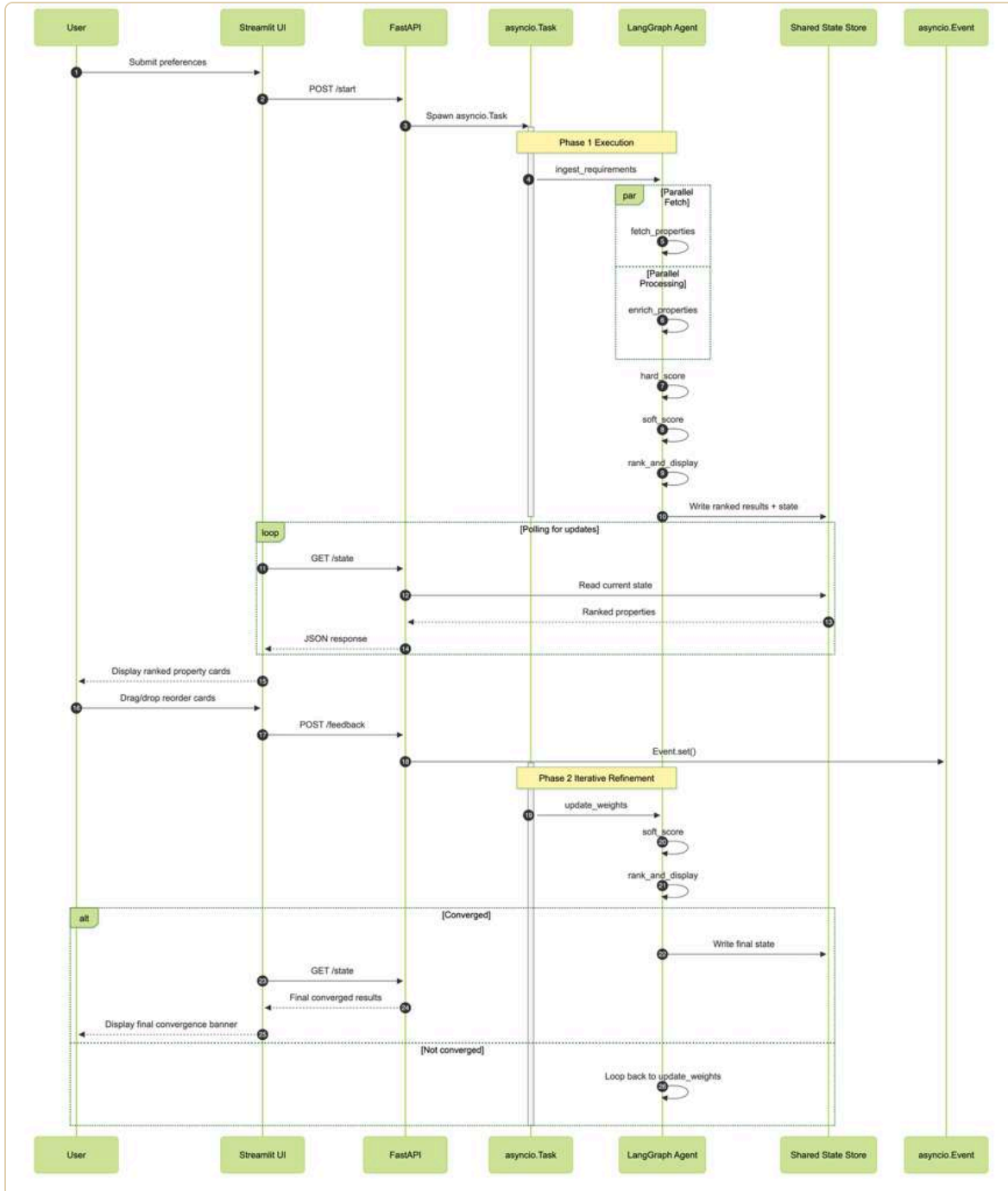


Figure 3. Full request-response and async-event sequence across the Streamlit-FastAPI-LangGraph boundary.

The graph execution is split into two phases to allow incremental result delivery. Phase 1 (Nodes 1–6) runs to completion and writes the initial ranked list to state before pausing for feedback. Phase 2 (Nodes 8, 5, 6) executes

once per feedback round. This split means Streamlit can display the first ranking as soon as Phase 1 completes, without waiting for the full feedback loop to terminate.

8. STREAMLIT FRONTEND

The frontend is a multi-page wizard built entirely in Streamlit, structured as a seven-page sequential flow (pages 0–6) followed by a results page (page 7). Page state is stored in `st.session_state` and persists across reruns. Navigation is handled by a page index that each button increments or decrements; the Streamlit script routes to the appropriate page function based on this index.

Page	Purpose	Key Inputs
0 — Welcome	Landing page with feature summary	"Get Started" button
1 — Location	Search area specification	City, state, optional ZIP, search radius slider
2 — Commute	Commute destination (optional)	Toggle, destination text input, max-minutes slider
3 — Property	Physical property requirements	Property type, bedroom count, bathroom count (radio buttons)
4 — Budget	Monthly rent range	Dual-handle slider (\$300–\$10,000)
5 — Preferences	Lifestyle preferences	Pet-friendly toggle, parking toggle, amenity multiselect
6 — Review	Requirements summary + search launch	"Search for Properties" button → POST /start
7 — Results	Property cards + feedback UI	Drag-and-drop reorder → POST /feedback

Table 6. Streamlit page structure and key inputs collected at each step.

The results page (Page 7) has two display modes. While the backend is processing Phase 1, an animated CSS house-building sequence is displayed — a purely cosmetic loading indicator that sets user expectations for the pipeline's approximate duration (typically 10–20 seconds for 15 enriched properties). Once results arrive, the page switches to a 5×2 property card grid showing address, city, rent, bedroom/bathroom count, a color-coded match percentage badge, and the LLM-generated rationale in italics below each card.

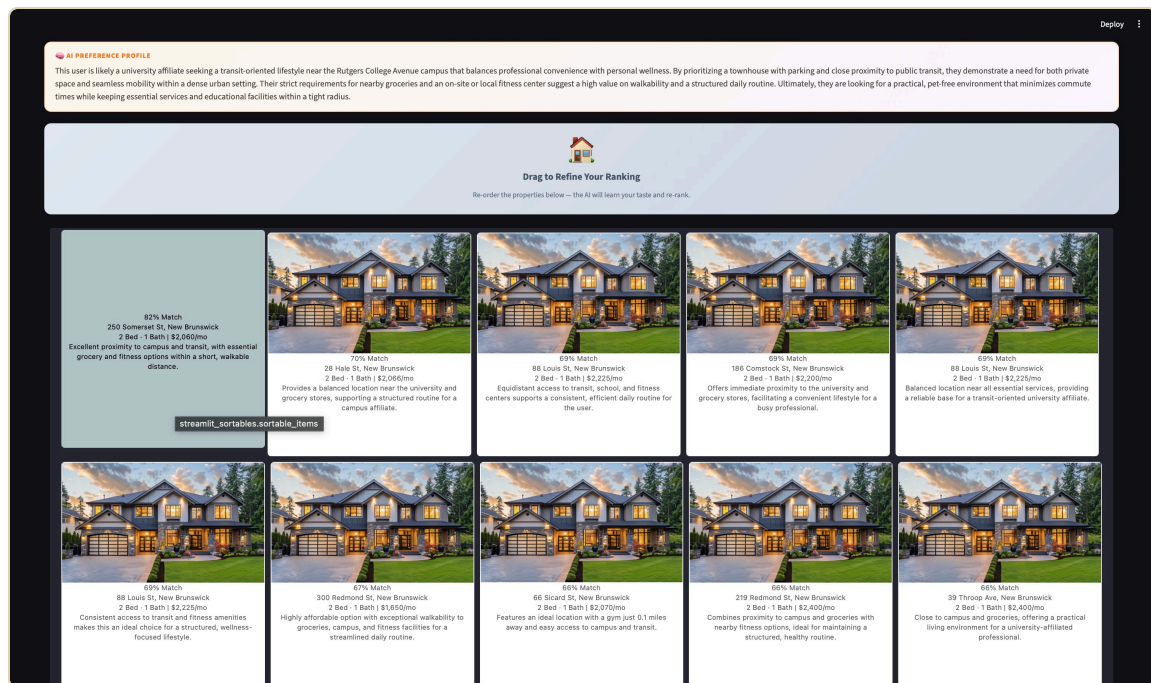


Figure 4. The drag-and-drop feedback interface. The AI Preference Profile card at the top shows the LLM's inferred understanding of the user's needs. The 5×2 property grid below presents the initial ranked results with match percentages and LLM-generated rationales, ready for the user to reorder.

When the backend signals `awaiting_feedback: true`, the grid is replaced by the drag-and-drop interface powered by the `streamlit-sortables` package. Properties are rendered as draggable cards; the user reorders them to express their preferences, then clicks "Submit My Ranking." The Streamlit app collects the ordered list of property IDs from the sortable component and POSTs them to the `/feedback` endpoint. A "Submit My Ranking" button finalizes the reordering and clears the displayed list to show the loading state while Phase 2 executes.

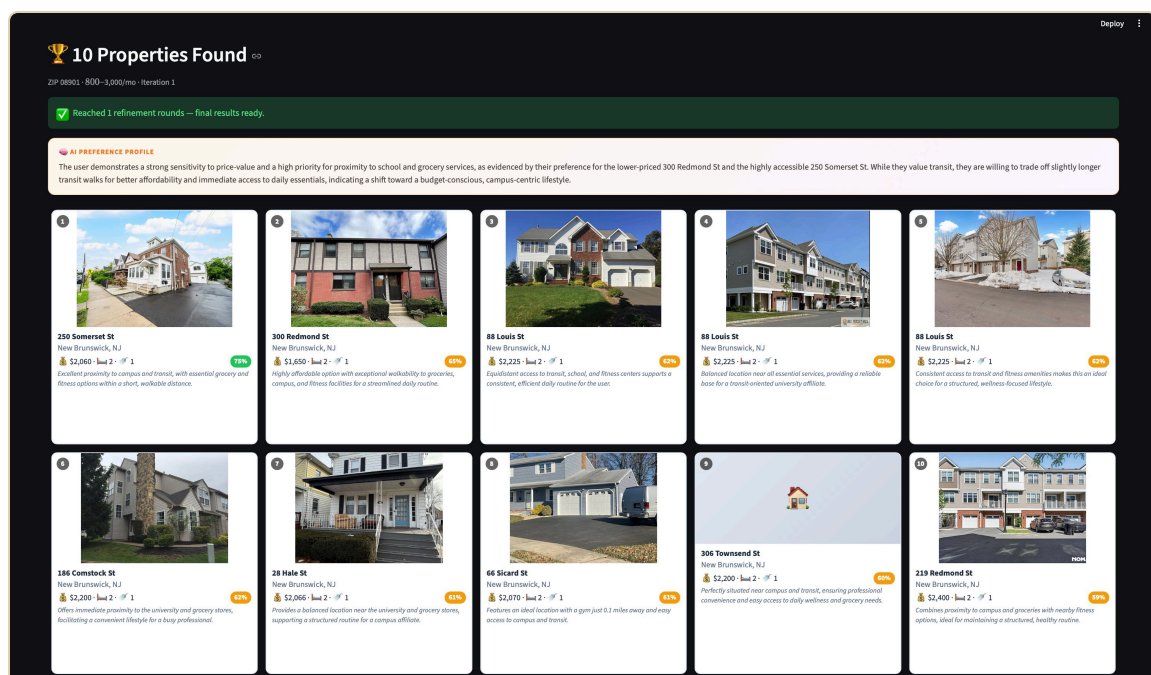


Figure 5. Final ranked results after convergence. The convergence banner confirms the feedback loop has terminated. The updated AI Preference Profile reflects what the system learned from the user's drag-and-drop reordering, and property cards now display real listing photographs alongside revised match scores.

9. EXTERNAL APIS AND DATA SOURCES

9.1 RentCast API

RentCast provides access to active long-term rental listings in the United States via a REST API. The system calls the `/v1/listings/rental/long-term` endpoint with parameters derived from the user's requirements, including city, state, ZIP code (if provided), price range, bedroom count, bathroom count, and radius in miles. Responses include address, rent, bedrooms, bathrooms, square footage, property type, and GPS coordinates — the full set of fields needed by both the hard scorer and the ArcGIS enrichment step.

The choice to use RentCast rather than scraping Zillow or Apartments.com reflects a deliberate engineering decision: scraping rental portals at query time violates their terms of service, introduces fragility when their HTML changes, and creates latency spikes that would break the user experience. RentCast provides a purpose-built API with a free tier sufficient for a class-scale project.

9.2 ArcGIS Places API

ArcGIS Places provides point-of-interest search via a REST endpoint that accepts latitude/longitude and a search text string, returning nearby places within a specified radius. The system uses the `/places/near-point` endpoint with a 5,000-meter radius, querying four amenity types per property using plain English search strings ("grocery store", "elementary school", "gym fitness", "bus stop").

This text-based search approach replaced an earlier category-ID-based approach that returned empty results for many locations. The plain-text search is both more robust and more intuitive to configure. The API returns place results sorted by distance; the system extracts only the nearest result per category and converts the distance from meters to miles.

API	Endpoint	Data returned	Tier used
RentCast	<code>/v1/listings/rental/long-term</code>	Active rental listings: address, rent, beds, baths, sqft, coordinates	Free (limited monthly calls)
ArcGIS Places	<code>/places-service/v1/places/near-point</code>	Distance in miles to nearest grocery, school, gym, transit stop	Free developer tier

API	Endpoint	Data returned	Tier used
Google	Via LangChain	Preference profiles, rationale	Free preview (gemini-3-flash-
Gemini	ChatGoogleGenerativeAI	strings, updated weight dicts	preview, gemini-3.1-flash-lite-preview)

Table 7. External APIs and their roles in the pipeline.

10. PROMPT ENGINEERING

Each of the three LLM nodes uses a different prompting strategy appropriate to its task. The following subsections describe the key design decisions for each.

10.1 Requirement Ingestion Prompt (Node 1)

The ingestion prompt is a zero-shot instruction to act as a "rental requirements analyst." It provides the full structured requirements JSON and the optional free-text comment, then asks the model to perform two tasks: identify latent preferences implied by the inputs, and write a concise preference profile. The prompt explicitly requests no preamble and no formatting — just the profile text — to avoid post-processing overhead. Temperature is set to 0.2 to ensure consistency across similar inputs.

10.2 Rationale Generation Prompt (Node 5)

Rationale generation uses a single batched prompt that provides the preference profile and all property details in a structured list, then requests a JSON object mapping property IDs to rationale strings. The 20-word limit per rationale is enforced in the prompt text ("write at most 20 words") rather than through token constraints. The instruction to avoid filler phrases ("This property offers") pushes the model toward specific, informative language. The response is parsed as JSON after stripping markdown code fences, with a fallback to empty strings if parsing fails.

10.3 Weight Update Prompt (Node 8)

The weight update prompt is the most structurally complex. It provides three context blocks: the current preference profile, the current weight dictionary (as JSON), and a ranked comparison showing each property's system rank, user rank, address, rent, and amenity distances. The model is asked to infer preference changes and produce a structured JSON response containing the updated profile, updated weights, and a one-sentence reasoning summary. The weights are renormalized to sum to 1.0 after parsing, providing a safety net against LLM floating-point imprecision.

Temperature is set to 0.2 for this node — lower than for creative writing tasks — because the desired output is an analytically grounded inference about preference ordering, not a creative or varied response. The structured

JSON output format with explicit field names further constrains the model's degrees of freedom and reduces parsing failures.

11. TECH STACK

Component	Technology	Role
Agent framework	LangGraph	Stateful directed graph with conditional edges and feedback loop
LLM interface	LangChain ChatGoogleGenerativeAI	Wraps Gemini API calls for Nodes 1, 5, and 8
LLM models	gemini-3-flash-preview, gemini-3.1-flash-lite-preview	Richer reasoning (Node 1, batch rationale) vs. fast structured output (Node 8)
Backend server	FastAPI + Uvicorn	Async HTTP bridge between Streamlit and LangGraph
Frontend	Streamlit	Multi-page requirements wizard and result display
Drag-and-drop	streamlit-sortables	User feedback interface for property reordering
Listing data	RentCast API	Active long-term rental listings
Amenity data	ArcGIS Places REST API	GPS-based proximity to grocery, school, gym, transit
Concurrency	asyncio + ThreadPoolExecutor	Parallel node execution (Nodes 1+2) and parallel enrichment (15 workers)
Numerical ops	NumPy	Cosine similarity convergence check, score normalization
Configuration	python-dotenv + .env file	API key management (RENTCAST_API_KEY, ARCGIS_API_KEY, GEMINI_API_KEY)

Table 8. Full tech stack with component roles.

12. DISCUSSION AND LIMITATIONS

12.1 Strengths of the Approach

The dual hard/soft scoring architecture is the most important design decision in this project. By separating objective criteria (which should not change based on user preference) from preference-based criteria (which should), the system avoids a common failure mode of pure LLM-based ranking: inconsistency and hallucination. The hard scorer is fully deterministic; the soft scorer is LLM-influenced only at the weight level, not at the

individual score level. A user who wants a two-bedroom apartment will not be shown a studio regardless of how the soft weights shift.

The choice to re-enter the feedback loop at Node 5 rather than Node 2 is equally important for practical usability. Re-fetching properties from RentCast on every feedback iteration would add 2–5 seconds of latency per round, exhaust API quotas quickly, and potentially change the candidate pool between iterations — making it difficult for the user to understand why rankings shifted. By fixing the candidate pool after Phase 1, the system ensures that every ranking shown to the user is over the same set of properties, and that changes in ranking are attributable solely to updated preference weights.

12.2 Limitations

The system caps the feedback loop at three refinement rounds. During the course demonstration this was temporarily reduced to one to ensure the session completed within the allotted time, but the production configuration allows up to three iterations. Even so, three rounds may not be sufficient to fully capture complex or multi-dimensional user preferences, particularly for users whose priorities shift significantly between iterations. A more adaptive cap — determined dynamically by weight stability rather than a fixed integer — would be preferable in a longer-horizon deployment.

The quiet neighborhood proxy — using gym distance as a surrogate for neighborhood noise — is a crude approximation. Gyms are not reliably co-located with high-traffic areas, and many quiet neighborhoods have nearby gyms. A more robust proxy would use pedestrian traffic density data, which is available through some paid map APIs but not through the free ArcGIS tier used here.

Property images are sourced from a local folder of representative stock photographs rather than from the actual listing. RentCast's API does not reliably return listing-specific images for all markets, and the planned RentCafe scraping component was deferred due to the absence of a public API. The visual representation of results is therefore illustrative rather than accurate, which reduces the realism of the UI but does not affect the scoring or ranking logic.

Finally, the in-memory session state means that all session data is lost if the FastAPI server restarts. For a production deployment, replacing `agent_states` and `feedback_events` with Redis or another persistent store would be the first required upgrade.

12.3 Future Work

Several natural extensions follow from the current architecture. Adding a commute-time scoring feature — using a routing API to compute actual drive time from each property to the user's workplace — would substantially improve relevance for users who commute. The commute destination is already collected in the Streamlit wizard (Page 2) and stored in the requirements object; only the Node 3 enrichment step and a new soft-score feature would need to be added.

A richer convergence check could incorporate Kendall's tau between the system and user rankings, rather than a simple equality test, giving the system a scalar measure of how much the two orderings agree rather than a binary match/no-match signal. This would allow the system to detect partial convergence — situations where the top-3 results agree even if the full ordering does not — and potentially stop the loop earlier.

13. CONCLUSION

This project builds an AI rental property recommendation agent that replaces the keyword-filter paradigm of traditional search platforms with a stateful, LLM-driven preference learning loop. The system's central contribution is the integration of three LLM roles — latent preference extraction, preference-aligned rationale generation, and implicit feedback interpretation — inside a LangGraph pipeline that manages state, parallelism, and convergence explicitly.

The dual hard/soft scoring architecture ensures that objective criteria anchor the ranking while learned preferences personalize it. The feedback loop that re-enters at the soft-scoring node rather than the property-fetch node ensures that preference learning is fast and API-efficient. The FastAPI bridge with `asyncio.Event` synchronization allows the long-running agent to coexist with Streamlit's synchronous execution model without blocking the user interface.

The system demonstrates that LLMs can play a substantive role in preference elicitation and learning — not just as a conversational interface, but as an inference engine that reads implicit signals from user behavior and updates a structured model of their taste. As rental data APIs improve and LLM inference costs continue to fall, this architecture scales naturally to longer feedback loops, richer feature sets, and more nuanced convergence criteria.

REFERENCES

- Chase, H., et al. (2022). LangChain. *GitHub repository*. <https://github.com/langchain-ai/langchain>
- LangChain AI. (2024). LangGraph: Build stateful, multi-actor applications with LLMs. *GitHub repository*. <https://github.com/langchain-ai/langgraph>
- Google. (2024). Gemini API Documentation. *Google AI for Developers*. <https://ai.google.dev/gemini-api/docs>
- RentCast. (2024). RentCast API Reference — Rental Listings. <https://developers.rentcast.io/reference/rental-listings>
- Esri. (2024). ArcGIS Places Service — Near-point API. *ArcGIS Developer Documentation*. <https://developers.arcgis.com/rest/places/near-point/>
- FastAPI. (2024). FastAPI documentation. <https://fastapi.tiangolo.com>
- Streamlit Inc. (2024). Streamlit documentation. <https://docs.streamlit.io>
- Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2), 235–256.
- Ziegler, D. M., et al. (2019). Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*.

